

Computational Geometry and Visibility Determination in legacy WMO Architectural Frameworks for Map Editor Remediation

The rendering of large-scale, intricate structures within the World Map Object (WMO) format—the primary architectural container for World of Warcraft during the Wrath of the Lich King (3.3.5a) era—requires a sophisticated understanding of spatial partitioning and topological connectivity. Developers of modern 3D map editors often encounter the "tall wall" phenomenon, an artifact where interior geometry, such as shop walls or dungeon corridors, becomes visible from a bird's-eye perspective, penetrating the exterior shell of the building. This issue arises from a reliance on simple bounding volume heuristics rather than the recursive portal-frustum clipping and Binary Space Partitioning (BSP) localization utilized by the original game client. To resolve these visual discrepancies, one must dissect the technical relationship between the WMO root file, the individual group files, and the metadata extracted by server-side toolsets like TrinityCore's vmap generator.

Structural Taxonomy of the World Map Object Format

The WMO format is fundamentally a cell-and-portal system designed to balance the high-fidelity requirements of indoor environments with the expansive nature of an open-world engine. Unlike M2 models, which are typically individual props or characters, WMOs are static environmental structures that can encompass entire cities, multi-level dungeons, or complex transition tunnels. The architectural design is split into two primary file types: the root file (.wmo) and the group files (_NNN.wmo), where NNN is a zero-padded index.¹

The root file serves as the topological master, containing metadata that defines how the individual groups interact with one another. It contains the MOHD (Header) chunk, which dictates global properties such as the number of groups, portals, and lights, as well as the overall bounding box of the structure.² Beneath this, the root file organizes material references (MOMT), texture paths (MOTX), and light definitions (MOLT). Most critically for visibility logic, the root file defines the physical boundaries between rooms via the MOPV (Portal Vertices) and MOPT (Portal Definitions) chunks.¹

Each group file contains the actual geometric mesh for a specific sector of the building. These files are split into various chunks, including MOGP (Group Header), MOVV (Vertices), MOVI (Indices), and MONR (Normals). The MOGP chunk contains flags that indicate if a group is an exterior shell, an interior room, or a transition zone.¹ The relationship between these chunks is

summarized in the table below.

Chunk Identifier	Location	Primary Function in Visibility Pipeline
MOHD	Root	Global structure metadata and WMO-wide bounding volume.
MOGI	Root	Information on group indices and their local bounding boxes.
MOPV	Root	List of vertices forming the planar polygons of portals.
MOPT	Root	Definition of portals, including normal vectors and connected groups.
MOPR	Root	Cross-reference table mapping groups to specific portals in MOPT.
MOGP	Group	Group-specific flags (Exterior, Interior, Show Skybox).
MOBN	Group	BSP tree nodes used for camera localization and collision.
MOBR	Group	BSP face indices mapped to specific triangles in the group.

The root file acts as the orchestrator of the structure.¹ By referencing the MOPR chunk, the engine can determine exactly which portals are accessible from any given group, facilitating the recursive visibility traversal that prevents interior geometry from "leaking" into the exterior

view.¹

The Camera Localization Mechanism: BSP Tree Traversal vs. Bounding Volumes

A frequent misconception in the development of map editors is that the "inside/outside" state is determined by whether the camera is within a group's Axis-Aligned Bounding Box (AABB). In the 3.3.5a client, AABB checks are used strictly for coarse-grained culling during the initial frustum pass. The definitive determination of the camera's current group is achieved through a point-in-polytope test facilitated by the Binary Space Partitioning (BSP) tree found in the MOBN and MOBR chunks of each WMO group file.⁴

Mathematical Foundations of WMO Localization

The BSP tree recursively divides the spatial volume of a group into half-spaces. Each node in the MOBN chunk represents a partitioning plane defined by the mathematical plane equation:

$$f(x, y, z) = Ax + By + Cz + D = 0$$

Where (A, B, C) represents the normal vector of the plane and D is the offset from the origin. To locate the camera, the engine performs a recursive traversal starting at the root node of the WMO group's tree. At each node, the camera's coordinates (x_c, y_c, z_c) are plugged into the plane equation.⁶

If $f(x_c, y_c, z_c) > 0$, the camera is considered to be in the "front" half-space, and the traversal proceeds to the front child node. Conversely, if $f(x_c, y_c, z_c) < 0$, the camera is in the "back" half-space. This process continues until a leaf node is reached. The leaf nodes of a WMO group represent specific convex subspaces. If the camera reaches a leaf node that is not flagged as "solid," and this leaf is contained within the logical volume of a group, the engine marks the camera as being "inside" that group.⁷

This method is substantially more robust than AABB tests because it accounts for complex, non-orthogonal architectural geometry. In cities like Stormwind, where buildings are often "kit-bashed" together and may have overlapping bounding boxes, the BSP traversal provides a mathematically certain way to identify the current environment.¹⁰ If the camera does not fall within the BSP volume of any loaded group, it is classified as being in the "Global Exterior."

Integrating BSP Traversal into Map Editors

For a map editor to correctly render WMOs from a bird's-eye view, it must first implement this localization logic. When the camera is high above a city, it will fail all BSP-in-volume tests,

placing it in the Global Exterior. This state serves as the "root" of the visibility graph. The editor should then only consider groups flagged with MOGP_EXTERIOR (0x8) as immediately visible. Interior groups (those lacking the 0x8 flag or explicitly flagged as 0x2000 interior) should be considered "occluded by default" unless they can be reached via a visible portal.²

The Mathematics of Portal-Based Visibility Traversal

The artifact of interior walls sticking up like tall slabs occurs because the editor is likely rendering all groups that fall within the view frustum regardless of their topological connectivity. To achieve parity with the World of Warcraft client, a recursive portal traversal algorithm is required. This system treats the WMO as a graph where groups are nodes and portals are the edges connecting them.¹²

The Recursive Clipping Algorithm

When the camera is outside a WMO, the "Exterior World" is the starting cell. The visibility pipeline proceeds as follows:

1. **Initial Pass:** Identify all WMOs whose global bounding boxes (from the MOHD chunk) intersect the view frustum.
2. **Exterior Submission:** Submit all groups flagged with MOGP_EXTERIOR (0x8) that are within the frustum for rendering.
3. **Portal Identification:** For each rendered exterior group, retrieve the list of portals from the MOPR and MOPT chunks.¹
4. **Portal Visibility Test:** Project the vertices of each portal polygon (from MOPV) into screen space.
 - If the camera is at a bird's-eye view, the doorway portal to a shop is typically pointing horizontally. From a steep overhead angle, the normal vector of the portal polygon is nearly perpendicular to the view vector.
 - The portal's polygon will either be back-facing or its projected screen area will be negligible.¹³
5. **Frustum Restriction:** If a portal is visible, the current view frustum is clipped against the planes formed by the camera position and the edges of the portal polygon. This generates a new, highly restricted frustum.¹³
6. **Recursive Discovery:** Recurse into the interior group (neighbor group) using the restricted frustum. Only the geometry within that group that falls inside the restricted "cone" of the portal is submitted to the GPU.

In the case of the bird's-eye view, the portal at the doorway fails the visibility test or results in a zero-volume restricted frustum. Consequently, the interior shop group—and its tall collision walls—is never added to the render queue. This naturally prevents the artifacts without requiring shader-based fragment discarding.¹²

BFS vs. DFS in Portal Traversal

While the original request mentions a Breadth-First Search (BFS) distance from exterior groups, the client often uses a Depth-First Search (DFS) for the recursive clipping pass to minimize the amount of memory needed for frustum stacks. However, the BFS distance is a valuable optimization for map editors. If the camera is more than a certain distance from a WMO, the editor can skip traversal for any group with a BFS distance greater than 1, as it is physically impossible to see into secondary interior rooms from a distance through a series of narrow openings.¹⁴

In-Depth Analysis of MOGP Flags and Group Meta-Data

The MOGP chunk contains a bitmask of flags that are the cornerstone of the WMO visibility and lighting system. Understanding these flags is essential for troubleshooting why certain interior shops should remain visible from the outside while others should be culled.

Flag Value	Name	Rendering and Visibility Impact
0x1	HAS_MOBN_MOBR	Confirms the presence of BSP data. Essential for camera localization.
0x2	HAS_MOCV	Indicates prebaked vertex colors (MOCV) for interior lighting.
0x4	SHOW_SKYBOX	Forces the skybox to render. Used in transition tunnels.
0x8	EXTERIOR	Identifies the group as part of the outer world shell.
0x40	CULL_OBJECTS	Enables doodad (M2) culling within the group volume.
0x80	HAS_MOLR	Presence of local light

		references (MOLR chunk).
0x100	HAS_MODR	Presence of local doodad references (MODR chunk).
0x200	HAS_MLIQ	Presence of liquid data (MLIQ). Relevant for interior pools.
0x400	HAS_MORB	Provides mapping between triangles and BSP leaves for efficient intersection.
0x2000	EXTERIOR_LIT	Forces the group to use outdoor lighting (Sun/Sky) instead of MOCV colors.

The Paradox of MOGP_EXTERIOR_LIT (0x2000)

The user's query highlights a conflict where interior shops should be visible from the outside. These groups are often flagged as 0x2000 (EXTERIOR_LIT). This flag serves as a hint to the engine: "Even if this group is physically indoors and reachable via a portal, light it using the outdoor world's parameters (sunlight, fog, and skybox)".²

In Stormwind shops, the 0x2000 flag is applied because these areas often have high ceilings or large open doors that allow significant natural light penetration. From a visibility standpoint, the client treats these as "Transition Groups." When the camera is outside, these groups are still strictly subject to portal culling. However, because they are lit similarly to the exterior, the engine can blend their vertices more seamlessly with the outdoor world.¹⁰

Transition Zones and MOGP_SHOW_SKYBOX (0x4)

The 0x4 flag is vital for structures like the Ironforge entrance or the Orgrimmar gate tunnels. When the camera enters such a group, the engine continues to render the global skybox behind the WMO geometry.² From an aerial perspective, groups with the 0x4 flag should often be prioritized for rendering because they represent the "entryways" into the structure. If a map editor hides these groups aggressively, the result is a visible gap or a solid black texture at the building's entrance.¹⁰

Topological Rendering from an Exterior Perspective

When the camera is outside a WMO, the standard procedure is not "render exterior groups, hide everything else." Instead, the client renders the exterior groups and then uses the portals to *discover* which interior groups might be partially visible.

Handling the Stormwind City Layout

In Stormwind, many buildings are constructed using a technique known as kit-bashing, where a standard exterior shell WMO is combined with specific interior room groups. This often leads to geometry intersections that are never intended to be seen by the player.¹⁰

1. **The Shell (MOGP_EXTERIOR):** This group contains the roof, the external stone walls, and the chimney. It is always rendered when the camera is in the city.²
2. **The Shop Interior (MOGP_INTERIOR/EXTERIOR_LIT):** This is a separate group that contains the wooden floor, the internal shop walls, the counter, and the ceiling. The floor of the shop is often placed exactly at the same Z-height as the exterior's ground plane, or slightly higher (0.01 units) to prevent Z-fighting from the ground.¹⁰
3. **The Artifact:** If the map editor renders both without portal clipping, the shop's interior walls—which are often modeled to be significantly taller than the room height to ensure they don't have gaps—will poke through the roof of the exterior shell.

The "tall wall" artifact is therefore not a flaw in the mesh but a lack of visibility logic. The interior shop group should *never* be submitted to the render queue unless the camera can "see" through the doorway portal.¹²

Facial Facades and Facade Culling

Some WMOs in Stormwind, notably the Cathedral of Light, use facades. A facade is a low-resolution representation of a building that is visible from a distance, which is then replaced by the actual WMO groups when the player gets closer.¹⁰ The map editor must check the distance-based Level of Detail (LOD) settings found in the WDL or WDT files to determine if it should even be rendering the full WMO groups or if a silhouette facade is more appropriate for the current zoom level.²⁵

The Nexus of Connectivity: MOPT, MOPR, and MOPV Chunks

The root file's portal data is the roadmap for all visibility decisions. To correctly implement portal culling, the editor must parse and transform this data into world space.

MOPT (Portal Definitions)

Each entry in the MOPT chunk defines a portal between two specific groups. It includes a normal vector (n) and an index into the MOPV (Vertices) chunk. The portal's orientation is

critical: the normal vector should ideally point from the base group into the neighbor group.¹

MOPR (Portal References)

The MOPR chunk is a flat array of indices that map to MOPT entries. Each group defined in the root file has a pointer to a start index and a count within the MOPR array. This tells the engine: "If the camera is in Group X, these are the only portals you need to check for visibility".¹

Root Chunk	Sub-Structure	Implementation Requirement
MOPV	float x, y, z	Must be transformed by the WMO instance's world matrix.
MOPT	uint16_t baseGroup, neighborGroup	Used to build the adjacency graph.
MOPR	uint16_t portalIndex	Maps the local group to global portal IDs.

By utilizing MOPR, the editor can efficiently determine connectivity. If the camera is outside (Group -1/Global Exterior), it should iterate through all groups marked as EXTERIOR (0x8) and then use their MOPR lists to find potential doorways into the interior rooms.¹²

Remediation Strategies for Interior Geometry Leakage

The current approach of using $|N.z| < 0.3$ to discard fragments in the pixel shader is a fragile heuristic that fails on complex geometry. It is particularly vulnerable to:

- **Sloped Architecture:** Many buildings in Azeroth feature angled walls or buttresses that fall outside the $\pm 17^\circ$ vertical threshold, allowing the artifact to remain visible.
- **Floating Floors:** Discarding walls often leaves the horizontal floor of an interior shop visible. From a bird's-eye view, these appear as random wooden or carpeted slabs floating over the stone streets.¹⁰
- **Jagged Edges:** Screen-space fragment discarding creates aliases and jagged edges on the silhouettes of the structures.²⁹

Technical Recommendation: The Topological approach

The editor should move away from fragment-level hacks and toward a geometric culling pass.

1. **Localization Pass:** At the start of the frame, traverse the BSP trees (MOBN) of all WMO instances near the camera. If the camera is not found in any convex leaf, it is marked as "Outdoor".⁵
2. **Exterior Seed:** Populate the render list with all groups where MOGP.flags & 0x8.
3. **Portal Search:** For each exterior group, fetch its portals from MOPR.
4. **Angle-Based Culling (Software):** Calculate the dot product between the portal's normal n and the camera's view vector v .
 - If $n \cdot v \geq 0$, the portal is back-facing to the camera.
 - At bird's-eye view, most doorway portals are back-facing or at extreme oblique angles.
5. **Frustum Clipping:** Clip the view frustum against the visible portals. For an aerial view, this clipping should result in the interior groups being culled because they fall outside the narrow "cone" of the visible doorway.¹³

This approach is 100% accurate to the original game's visual intent and naturally resolves all "tall wall" and "floating floor" artifacts without needing to manually flag individual groups.¹⁰

Integration of TrinityCore VMAP Logic into Rendering Pipelines

TrinityCore's vmap data provides the same underlying BSP and portal information as the original WMO files, but it is optimized for server-side calculations. When using vmaps for a map editor, one must be aware of how the extractor handles liquid data and vertex alpha.³⁰

VMAP Tree Traversal

TrinityCore uses a .vmtree and .vmtile system to manage large spatial datasets. The LineOfSight and getHeight functions in the core use a highly optimized version of the BSP traversal described earlier.³³ For a map editor, it is often more performant to use these pre-extracted trees to identify which WMO instances are relevant to the camera before even loading the detailed group geometry.

Handling Interior vs. Outdoor Spells

In the TrinityCore engine, the Map::IsOutdoors() function relies on the MOGP flags. Specifically, it checks for 0x8 (EXTERIOR) and 0x2000 (EXTERIOR_LIT). This mirrors how the client chooses which lighting model to apply.¹⁹

Spell/Logic Context	Requirement	MOGP Flag Checked
---------------------	-------------	-------------------

Mount Usage	Must be Outdoors	0x8 (EXTERIOR) or 0x4 (SKYBOX). ¹¹
Indoor Lighting	Prebaked MOCV	0x2 (HAS_MOCV) and NOT 0x2000.
Window Glows	Self-Illumination	0x10 (SIDN Scalar). ²
Collision LOS	Server-side LoS	MOPY Collision bits. ²

The editor can leverage this logic to color-code groups. For example, groups that allow mounting can be rendered with full brightness, while interior-only groups are culled or rendered with dark vertex tints to signal they are "inside".²

Case Study: Solving the Stormwind City Artifact Paradox

Stormwind is notoriously difficult to render correctly in custom tools because of its heavy use of overlapping WMOs and decorative interior rooms that lack exterior-facing geometry.

The Trade District Analysis

In the Trade District, we find several shops that are essentially wooden boxes placed inside stone exterior shells.

- **The Stone Shell:** Marked as MOGP_EXTERIOR (0x8). It has the roof and the outer walls.
- **The Wood Shop:** Often marked as MOGP_EXTERIOR_LIT (0x2000) or simply lacks the 0x8 flag.

If the editor is in bird's-eye view:

1. The camera is at $Z = 150$, while the building is at $Z = 80$.
2. The doorway portal is a rectangle from $(X, Y, 80)$ to $(X, Y, 85)$, pointing along the Y-axis.
3. The camera's view vector is nearly $(0, 0, -1)$.
4. The doorway portal's normal is $(0, 1, 0)$.
5. The dot product is 0, meaning the portal is perpendicular and barely visible.
6. The frustum clipping for the Wood Shop group will result in a near-zero area.
7. The Wood Shop is not rendered. No wooden walls poke through the stone roof.¹⁰

The Cathedral of Light Analysis

The Cathedral square features transition tunnels (portals) that lead between districts. These districts are often separate maps or logically isolated areas within the same world.¹⁰ Once the player passes through the tunnel into the Cathedral Square, the Trade District is no longer visible. This is achieved by the client "resetting" its visibility chain at the exit of the tunnel portal. A map editor that renders all districts simultaneously will experience massive performance degradation and overlapping geometry artifacts. Implementing the portal-frustum chain is the only way to replicate this "isolation" logic.¹⁰

Advanced Visibility: Vertex Alpha Attenuation and MOCV

For transitions that are not strictly binary (i.e., you are standing in a doorway looking into a dark room), the client uses vertex alpha attenuation. The `CMapObjGroup::FixColorVertexAlpha` function (WoTLK version) iterates through the MOCV chunk and modifies the alpha values of vertices near portals.²

In a map editor, if full portal-frustum clipping is too complex for the current development phase, a secondary strategy is to utilize this alpha data. Vertices in interior groups can be weighted by their distance to the nearest portal. As the camera moves further from the portal (e.g., zooms out to bird's-eye), the entire interior mesh can be faded to an alpha of 0.²⁵ This is a more visually pleasing fallback than the $|N.z|$ fragment discard, as it allows for smooth transitions and respects the geometry's slope.

Technical Resolution of Specific User Questions

1. **Localization Determination:** The WoW client determines "camera is inside" based on a recursive walk through the BSP tree (MOBN chunk) of the nearest WMO groups. If the camera point (X, Y, Z) lands in a non-solid leaf of a group's BSP, the camera is inside that group. It is *not* based on simple AABB checks, which are only used for coarse culling.⁵
2. **Exterior Rendering Logic:** When outside, the client renders all groups flagged with `MOGP_EXTERIOR (0x8)` that are in the frustum. It then looks through all portals associated with those groups. If a portal (e.g., a doorway) is visible from the camera's high-angle perspective, it will render the interior group reachable through it, but *only* clipped to that portal's restricted frustum. Since doorways are invisible from above, the interiors are naturally culled.¹²
3. **Hiding Non-Exterior Groups:** Hiding all non-exterior groups is a rough approximation but will fail for buildings where shops are interior groups. The correct approach is not to hide them globally, but to hide them *relative to the current camera cell*. From the Global Exterior cell, interior shops are only reachable through narrow portal polygons that will fail

the high-angle frustum test.¹⁰

4. **Relevant MOGP Flags:** 0x4 (SHOW_SKYBOX) is crucial for entryways. 0x2000 (EXTERIOR_LIT) is a lighting hint that often identifies interior areas intended to be partially visible from outside. 0x40 (CULL_OBJECTS) affects the visibility of small props within the volume.²
5. **Indoor/Outdoor Per Group:** There is a concept of "Force Indoors" (0x2) and "Force Outdoors" (0x4) in the WMOAreaTable DBC, which can override the group flags for certain game logic (like mounting or weather effects). However, for *rendering*, the portal graph is the absolute authority.¹⁹
6. **Portal Traversal from Outside:** Yes, the client *always* does portal traversal if the camera is near a WMO. It starts from the "Global Exterior" and treats all exterior (0x8) groups as the first layer of the graph. It then recurses through their portals into the interior. The "culling" happens because the recursion dies when the portals are not visible.¹²
7. **MOPT/MOPR Relationship:** MOPT defines the portal polygons and their destination groups. MOPR provides the range of portals for each specific group. The engine uses MOPR to find which MOPTs to check based on the camera's current group. This prevents the engine from checking every portal in a massive WMO like Stormwind, only those adjacent to the camera's current sector.¹

Future-Proofing Toolsets for Modern Rendering Paradigms

As map editors evolve, moving away from legacy hardware-emulation and toward modern rendering (e.g., Vulkan or DirectX 12), the need for efficient visibility determination becomes even more paramount. While modern GPUs can handle much higher triangle counts, rendering thousands of invisible interior rooms in a city like Dalaran or Stormwind still places unnecessary pressure on the vertex shader and memory bandwidth.

Implementing a true cell-and-portal graph within the editor's core engine not only solves the "tall wall" artifacts but also provides:

- **Accurate Collision Visualization:** By knowing which group the camera (or the user's cursor) is in, the editor can highlight relevant collision geometry while ghosting out irrelevant layers.³¹
- **Enhanced Lighting Previews:** Map editors can accurately preview how outdoor light bleeds into interiors by implementing the client's transition logic for MOCV and SIDN colors.²
- **Streamlined Workflow:** Users can toggle between "Client View" (full portal culling) and "Editor View" (render everything) to troubleshoot structural gaps in the WMO data.

In conclusion, the resolution to interior geometry artifacts in 3.3.5a map editors lies in a shift from fragment-level heuristics to a global, topological understanding of the WMO data structure. By integrating the root file's portal connectivity (MOPT/MOPR) with the group file's

spatial localization (BSP/MOBN), developers can eliminate artifacts while providing a toolset that reflects the mathematical elegance of the original engine's design. This technical synergy between vmap collision data and client-side visibility logic is the prerequisite for professional-grade environment manipulation and world-building.¹²

Works cited

1. wow-wmo - Lib.rs, accessed February 23, 2026, <https://lib.rs/crates/wow-wmo>
2. WMO - wowdev, accessed February 23, 2026, <https://wowdev.wiki/WMO>
3. WMO File - ClientFiles - getMaNGOS | The home of MaNGOS, accessed February 23, 2026, <https://www.getmangos.eu/wiki/referenceinfo/clientfiles/wmo-file-r20030/>
4. Visibility Algorithms II: Partitioning Trees - UT Austin Computer Science, accessed February 23, 2026, <https://www.cs.utexas.edu/~bajaj/graphics2013/cs354/lectures/lect24.pdf>
5. WMO/Rendering - wowdev, accessed February 23, 2026, <https://wowdev.wiki/WMO/Rendering>
6. Motivation for BSP Trees: The Visibility Problem, accessed February 23, 2026, <https://groups.csail.mit.edu/graphics/classes/6.838/F01/lectures/BSP/BSP2D.pdf>
7. BSP Tree FAQ, accessed February 23, 2026, <https://people.eecs.berkeley.edu/~jrs/274s19/bsptreefaq.html>
8. Efficient Object BSP Trees Navendu Jain, Sorav Bansal, Sanjiv Kapoor Computer Science and Engineering Department, Block VI, Indi - Microsoft, accessed February 23, 2026, <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/01/G46.pdf>
9. Binary space partitioning - Valve Developer Community, accessed February 23, 2026, https://developer.valvesoftware.com/wiki/Binary_space_partitioning
10. What happens to this blue roofed section when viewing the entrance of Stormwind from the rear? One way texture? : r/classicwow - Reddit, accessed February 23, 2026, https://www.reddit.com/r/classicwow/comments/144pbdv/what_happens_to_this_blue_roofed_section_when/
11. wowmapviewer/src/wmo.cpp at master - GitHub, accessed February 23, 2026, <https://github.com/cleverca22/wowmapviewer/blob/master/src/wmo.cpp>
12. Portal Culling - Panda3D Manual, accessed February 23, 2026, <https://docs.panda3d.org/1.9/cpp/programming/render-attributes/occlusion-culling/portal-culling>
13. Implementation Details | Immersive Portals - qouteall, accessed February 23, 2026, <https://qouteall.fun/immptl/wiki/Implementation-Details.html>
14. Optimizing recursive portal traversing - Game Development Stack Exchange, accessed February 23, 2026, <https://gamedev.stackexchange.com/questions/212384/optimizing-recursive-portal-traversing>
15. Portal-Based Culling Scene Manager Tutorial - Visualization Library, accessed February 23, 2026,

- https://visualizationlibrary.org/documentation/pag_guide_portals.html
16. Weekly : Portal Culling - 3D Programming - 3DKingdoms, accessed February 23, 2026, <https://3dkingdoms.com/weekly/weekly.php?a=26>
 17. UnrealWiki: BSP Tree - Beyond Unreal, accessed February 23, 2026, <https://beyondunrealwiki.github.io/pages/bsp-tree.html>
 18. Buildings often look nothing alike from the inside and outside in many cities of WoW - Reddit, accessed February 23, 2026, https://www.reddit.com/r/wow/comments/i7vzbp/buildings_often_look_nothing_alike_from_the/
 19. WMOAreaTable.dbc | TrinityCore MMo Project Wiki, accessed February 23, 2026, <https://trinitycore.info/files/DBC/335/wmoareatable>
 20. Where is The Culling Of Stratholme entrance - WoW WOTLK Classic - YouTube, accessed February 23, 2026, <https://www.youtube.com/watch?v=nWKXkSlghVM>
 21. House exterior not matching interior design - Blizzard Forums, accessed February 23, 2026, <https://us.forums.blizzard.com/en/wow/t/house-exterior-not-matching-interior-design/2208684>
 22. Never ever leave the birds-eye view : r/Workers_And_Resources - Reddit, accessed February 23, 2026, https://www.reddit.com/r/Workers_And_Resources/comments/1p4jilm/never_ever_leave_the_birdseye_view/
 23. Introduction to Occlusion Culling | by Umbra 3D - Medium, accessed February 23, 2026, <https://medium.com/@Umbra3D/introduction-to-occlusion-culling-3d6cfb195c79>
 24. Visibility calculation algorithms : r/gamedev - Reddit, accessed February 23, 2026, https://www.reddit.com/r/gamedev/comments/1j79i6/visibility_calculation_algorithms/
 25. Implement distance culling · Issue #162 · wowserhq/wowser - GitHub, accessed February 23, 2026, <https://github.com/wowserhq/wowser/issues/162>
 26. WDL/v18 - WowDev wiki, accessed February 23, 2026, <https://wowdev.wiki/WDL/v18>
 27. Kelsidavis/WoWee: World of Warcraft Engine Experiment - a custom opensource client. - GitHub, accessed February 23, 2026, <https://github.com/Kelsidavis/WoWee>
 28. Bird Eye Interior: Over 879 Royalty-Free Licensable Stock Illustrations & Drawings, accessed February 23, 2026, https://www.shutterstock.com/search/bird-eye-interior?image_type=illustration
 29. Rendering Artifacts in Perspective Full Overview - Q&A - HomeTalk Forum, accessed February 23, 2026, <https://hometalk.chiefarchitect.com/topic/629-rendering-artifacts-in-perspective-full-overview/>
 30. spawn_group_template | TrinityCore MMo Project Wiki, accessed February 23, 2026, https://trinitycore.info/database/master/world/spawn_group_template
 31. Core/Vmaps: vmaps for WMO · Issue #19185 · TrinityCore/TrinityCore - GitHub, accessed February 23, 2026,

<https://github.com/TrinityCore/TrinityCore/issues/19185>

32. Windows Server Setup | TrinityCore MMo Project Wiki, accessed February 23, 2026, <https://trinitycore.info/en/install/Server-Setup/Windows-Server-Setup>
33. Dbc, Maps, mmap, Vmap problems - Page 3 - Help and Support - TrinityCore, accessed February 23, 2026, <https://talk.trinitycore.org/t/dbc-maps-mmap-vmap-problems/13688?page=3>